WEB FUZZING MASTERY

The Ultimate Guide to FFUF & Gobuster

A Premium Operations Manual for Penetration Testers, Red Teamers & Bug Bounty Hunters

Author: Security AI Architect

Version: 2.0.1 (Enterprise Edition)

Table of Contents

SECTION 1 — INTRODUCTION

What is FFUF?

FFUF (Fuzz Faster U Fool) is a profoundly fast web fuzzer written in Go. Originally created to replace slower tools like DirBuster and WFUZZ, FFUF has evolved into the Swiss Army knife of web fuzzing. It is highly flexible, allowing testers to fuzz headers, POST data, JSON bodies, and URLs with exact precision using the `FUZZ` keyword.

What is Gobuster?

Gobuster is a directory/file, DNS, and VHost busting tool written in Go. While FFUF focuses on arbitrary HTTP request fuzzing, Gobuster is designed with specific operational modes (like `dir`, `dns`, `vhost`, `s3`). It is less flexible than FFUF regarding raw request manipulation but excels in out-of-the-box speed, simplicity, and specific enumeration tasks like DNS brute-forcing and cloud bucket hunting.

When to use FFUF vs Gobuster

Scenario	Recommended Tool	Reasoning
API Endpoint Fuzzing (POST/JSON)	**FFUF**	Native support for custom methods, raw HTTP requests, and arbitrary injection points.
Standard Directory Enumeration	**Either**	Both are incredibly fast. Gobuster is slightly easier for beginners; FFUF allows for more complex filtering.
DNS Subdomain Brute-forcing	**Gobuster**	Gobuster has a dedicated, highly optimized `dns` mode specifically for resolving subdomains.
AWS S3 / Google Cloud Buckets	**Gobuster**	Gobuster features dedicated `s3` and `gcs` modules for cloud asset discovery.
Complex Authentication / Proxy Chaining	**FFUF**	FFUF perfectly integrates with Burp Suite (`-x`) and supports complex session cookies/headers.

> ⚠ **Legal and Ethical Disclaimer**
>
> The tools and techniques documented in this manual generate significant network traffic and can be highly disruptive (causing Denial of Service or log exhaustion). **DO NOT** run FFUF or Gobuster against systems, networks, APIs, or applications for which you do not possess explicit, documented authorization. Misuse of these tools is a violation of international computer misuse laws.

SECTION 2 — INSTALLATION

Both tools are written in Go, which means they are distributed as static binaries, making installation trivial across operating systems.

Kali Linux / Debian / Ubuntu

```
sudo apt update
sudo apt install ffuf gobuster -y
```

Arch Linux (BlackArch)

```
sudo pacman -S ffuf gobuster
```

macOS (Homebrew)

```
brew install ffuf gobuster
```

Go Install (Recommended for Latest Versions)

Installing via Go ensures you have the absolute latest builds directly from the repositories.

```
go install github.com/ffuf/ffuf@latest
go install github.com/OJ/gobuster/v3@latest
```

Ensure `~/go/bin` is in your `$PATH`.

Docker

```
docker run --rm -v $(pwd):/data minamijoyo/ffuf -u https://target/FUZZ -w /data/
wordlist.txt
```

> **i Verifying Installation**
>
> Run `ffuf -V` and `gobuster version` to ensure successful installation. If you receive a "command not found" error after Go installation, add `export PATH=$PATH:$(go env GOPATH)/bin` to your `.bashrc` or `.zshrc`.

SECTION 3 — FFUF COMPLETE GUIDE

FFUF is controlled via a series of flags grouped into logical categories. Mastering these flags transitions a tester from a script kiddie to an advanced operator.

HTTP Options

These flags control how the HTTP request is constructed and sent.

- `-H` : **Header**. Add custom headers. Format: `"Name: Value"`. Useful for Auth, Cookies, or custom routing.
  Example: `ffuf -H "Authorization: Bearer JWT_TOKEN" -H "X-Forwarded-For: 127.0.0.1" ...`
- `-X` : **HTTP Method**. Change from the default GET. Supports POST, PUT, DELETE, OPTIONS, etc.
  Example: `ffuf -X POST -u https://api.target.com/users/FUZZ ...`
- `-d` : **Data**. The POST body data. Can be URL encoded, JSON, or XML.
  Example: `ffuf -X POST -d '{"username":"FUZZ", "password":"password123"}' -H "Content-Type: application/json" ...`
- `-x` : **Proxy**. Route traffic through Burp Suite or ZAP for inspection.
  Example: `ffuf -x http://127.0.0.1:8080 -u ...`
- `-c` : **Color**. Colorize the output (highly recommended for readability).
- `-r` : **Follow Redirects**. By default, FFUF does not follow 301/302 redirects. Use this to follow them, which is crucial for discovering internal paths.
- `-recursion` : **Recursive Fuzzing**. Automatically fuzzes newly discovered directories. Requires `-recursion-depth` to limit depth.
- `-request` : **Raw Request**. Provide a raw HTTP request file (e.g., saved from Burp) and fuzz inside it.

> 💡 **Pro Tip: The Raw Request Workflow**
>
> Instead of struggling to construct complex multi-part forms or JSON structures via CLI flags, intercept the request in Burp Suite, place the word `FUZZ` where you want to inject payloads, save it to a file (e.g., `req.txt`), and use:
> `ffuf -request req.txt -request-proto https -w payloads.txt`

Matcher Options (The "Whitelist")

Matchers tell FFUF *what* to show you. If a response matches these rules, it is printed to the screen.

Flag	Description	Example Usage
`-mc`	Match Status Code (Default: 200,204,301,302,307,401,403)	`-mc 200,403`
`-ms`	Match Response Size (Bytes)	`-ms 1543`
`-ml`	Match Amount of Lines in Response	`-ml 42`
`-mw`	Match Amount of Words in Response	`-mw 100`
`-mr`	Match Regular Expression within body	`-mr "root:x:0:0"`

Filter Options (The "Blacklist")

Filters are the inverse of Matchers. They tell FFUF what to *hide*. This is critical for **reducing noise** on targets that return wildcard 200 OK responses for everything.

- `-fc` : **Filter Code**. Hide specific status codes (e.g., `-fc 404,403`).
- `-fs` : **Filter Size**. Extremely useful for wildcard responses. If an invalid page always returns exactly 2341 bytes, filter it out: `-fs 2341`.
- `-fl` : **Filter Lines**. Filter by the number of lines in the response.
- `-fw` : **Filter Words**. Filter by word count. Excellent for WAF blocks that return a standard block page.

Input Options & Fuzzing Modes

FFUF supports multiple wordlists simultaneously, allowing for complex matrix attacks.

```
ffuf -w users.txt:USER -w passes.txt:PASS -u https://target.com/login?u=USER&p=PASS
```

When using multiple wordlists, you must specify a `-mode`:

- **sniper**: Default. Replaces one keyword at a time.
- **pitchfork**: Iterates through both lists simultaneously (User1:Pass1, User2:Pass2). Ideal for credential stuffing where lists map 1:1.
- **clusterbomb**: Tests every combination (User1:Pass1, User1:Pass2, User2:Pass1...). Ideal for comprehensive brute forcing.

SECTION 4 — FFUF PRACTICAL ATTACK SCENARIOS

1. Hidden Directory & File Discovery

```
ffuf -c -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt -u https://target.com/FUZZ -e .php,.txt,.bak,.zip -fc 404
```

Use Case: Finds standard directories and common file extensions. The `-e` flag appends the extensions to each wordlist entry.

2. Virtual Host (VHost) Discovery

Often, staging or dev environments are hosted on the same IP but require a specific `Host` header to route correctly.

```
ffuf -c -w subdomains.txt -u http://10.10.10.50 -H "Host: FUZZ.target.com" -fs 1250
```

Use Case: We filter size `1250` because that is the size of the default fallback web page. Any valid VHost will likely have a different size.

3. API Parameter Discovery (GET)

```
ffuf -c -w parameters.txt -u "https://api.target.com/v1/user?FUZZ=test" -mr "Invalid parameter"
```

4. POST JSON Fuzzing (Authentication Bypass/SQLi)

```
ffuf -c -X POST -u https://target.com/api/login -H "Content-Type: application/json" -d '{"email":"admin@target.com", "password":"FUZZ"}' -w passwords.txt -mc 200
```

5. Local File Inclusion (LFI) Hunting

```
ffuf -c -w lfi_payloads.txt -u "https://target.com/view.php?page=FUZZ" -mr "root:x:0:0:"
```

Workflow: We use a payload list containing strings like `../../../../etc/passwd`. The `-mr` (Match Regex) flag ensures we only see results where the server response actually contains the contents of the passwd file.

SECTION 5 — GOBUSTER COMPLETE GUIDE

Gobuster is heavily modularized. You must invoke a specific mode before applying flags.

1. `dir` Mode: Directory and File Enumeration

The staple mode of Gobuster.

```
gobuster dir -u https://target.com/ -w /usr/share/seclists/Discovery/Web-Content/common.txt -t 50 -x php,html,txt
```

- `-u` : Target URL.
- `-w` : Wordlist.
- `-t` : Threads (Default 10, push to 50-100 for speed).
- `-x` : Extensions to append.
- `-b` : Exclude status codes (e.g., `-b 404,403`).
- `-s` : Positive status codes (e.g., `-s 200,204,301`).

2. `dns` Mode: Subdomain Hunting

Incredibly fast concurrent DNS resolver.

```
gobuster dns -d target.com -w subdomains.txt -t 100 -i
```

- `-d` : The target domain.
- `-i` : Show IP addresses corresponding to the resolved subdomains.
- `-c` : Show CNAME records.
- `-r` : Use a custom DNS resolver (e.g., `-r 8.8.8.8`) to bypass local network DNS poisoning.

3. `vhost` Mode: Virtual Host Brute Forcing

```
gobuster vhost -u https://target.com -w subdomains.txt --append-domain
```

Note: The `--append-domain` flag automatically turns a wordlist of `dev` into `dev.target.com` in the Host header.

4. Cloud Modes: `s3` and `gcs`

Hunting for exposed public cloud storage buckets.

```
gobuster s3 -w bucket_names.txt
gobuster gcs -w bucket_names.txt
```

SECTION 6 — WORDLISTS & DICTIONARIES

A fuzzer is only as good as its wordlist. Sending 10 million irrelevant requests is worse than sending 10,000 highly targeted requests.

The Gold Standard: SecLists

Clone it immediately if you don't have it: `git clone https://github.com/danielmiessler/SecLists.git`

Goal	Recommended Wordlist
Initial Fast Directory Scan	`Discovery/Web-Content/raft-small-directories.txt`
Comprehensive Directory Scan	`Discovery/Web-Content/directory-list-2.3-medium.txt`
API Endpoints	`Discovery/Web-Content/api/api-endpoints.txt`
Subdomain Enumeration	`Discovery/DNS/subdomains-top1million-110000.txt`
Parameter Fuzzing	`Discovery/Web-Content/burp-parameter-names.txt`

> 💡 **Context-Aware Lists (Assetnote)**
>
> For modern Bug Bounty hunting, SecLists can sometimes feel dated. Use **Assetnote Wordlists** (wordlists.assetnote.io), which are compiled daily from internet-wide scans and GitHub repos. Use lists like `httparchive_apiroutes_2024.txt` for bleeding-edge API hunting.

SECTION 7 — ADVANCED TECHNIQUES & AUTOMATION

Tool Chaining & Recon Pipelines

Modern hacking relies on piping the output of one tool directly into FFUF. FFUF supports reading from Standard Input (stdin) by omitting the wordlist file entirely or using `-`.

Chain: Subdomain Discovery to Live Host to Fuzzing

Instead of manually scanning hundreds of subdomains, chain `subfinder`, `httpx`, and `ffuf` together in bash:

```
subfinder -d target.com -silent | httpx -silent -threads 100 | xargs -I@ ffuf -w common.txt -u @/FUZZ -mc 200 -t 50 -c
```

Explanation: Subfinder gets subdomains -> httpx filters only the ones with active web servers -> xargs passes each live URL to FFUF to search for standard exposed files.

Bypassing Rate Limits (WAFs)

If Cloudflare or a WAF blocks your aggressive scanning (HTTP 429 Too Many Requests or 403 Forbidden), utilize FFUF's rate limiting and delay capabilities:

- `-rate 5` : Only send 5 requests per second.
- `-p 0.5` : Pause 0.5 seconds between requests.
- `-H "X-Forwarded-For: 127.0.0.1"` : Spoof the origin IP (sometimes bypasses poor IP-based rate limiting).
- **Rotate User-Agents:** Pipe a list of user agents into FFUF or use a proxy rotator like Proxychains.

SECTION 8 — BUG BOUNTY WORKFLOWS

The 4-Step Master Content Discovery Methodology

Phase 1: Broad Recon (The Wide Net)

Use Gobuster DNS to find subdomains, then run a fast, light FFUF scan using `raft-small-words.txt` on every active subdomain. Goal: Find low-hanging fruit (e.g., `/admin`, `/api`, `/.git/`).

Phase 2: Deep Contextual Fuzzing

Based on Phase 1, if you find a target running WordPress, switch your wordlist to `wp-plugins.txt`. If you find an API, switch to `api-endpoints.txt`. **Do not use generic lists for deep fuzzing.**

Phase 3: Parameter Mining

Once you locate a functional endpoint (e.g., `/api/v1/users/profile`), use FFUF to look for hidden developer parameters (e.g., `?debug=true`, `?admin=1`):

```
ffuf -w parameters.txt -u "https://target.com/api/profile?FUZZ=1" -fs 450
```

Phase 4: Vulnerability Specific Fuzzing

Focus on specific bugs. Fuzz for SSRF via the `Host` header. Fuzz for XSS by injecting payloads into parameters and using `-mr` to check if they reflect unsanitized in the response.

SECTION 9 — TROUBLESHOOTING

Issue / Error	Root Cause	Solution / Flag to Fix
Everything is returning 200 OK	Server has a catch-all wildcard mechanism.	Use `-fs <size>` to filter the size of the wildcard page. Alternatively, use `-fw <words>`.
HTTP 429 Too Many Requests	Triggered the Web Application Firewall (WAF) rate limit.	Throttle your requests. Drop `-t` to 5, add `-p 1` (1 second pause), or use `-rate`.
Connection Refused / Timeouts	Target server is overwhelmed or blocking your IP.	Lower threads immediately. Use `-timeout 10` to wait longer for responses.
Results look messy and unreadable	Missing color or overlapping outputs.	Always use the `-c` flag. If redirecting to a file, use `-o output.json` for clean logging.

SECTION 10 — PERFORMANCE TUNING

To maximize efficiency, you must tune FFUF based on your network and the target's capacity.

- **VPS vs Local:** Always run heavy FFUF scans from a cloud VPS (e.g., DigitalOcean, AWS) rather than your home connection. A VPS has a massive bandwidth pipe and won't crash your local router's NAT tables.
- **Thread Count (`-t`):**
  - Fragile Internal Network: `-t 10` to `-t 20`
  - Standard Web Server: `-t 40` to `-t 50`
  - Enterprise / Cloud Hosted: `-t 100` to `-t 200`
- **Keep-Alive:** By default, FFUF attempts to reuse connections. If the server drops them, FFUF will stall. Use `-sa` (Stop on all errors) to kill the job if the server dies.

SECTION 11 — COMPARISON CHARTS

Feature Matrix	FFUF	Gobuster
Primary Language	Go	Go
Raw Request Fuzzing	✅ Yes (Burp integration)	❌ No
Dedicated DNS Resolver	❌ No	✅ Yes (Highly optimized)
Multiple Wordlists	✅ Yes (Clusterbomb/Pitchfork)	❌ No
S3/GCS Cloud Hunting	❌ No	✅ Yes
Auto-Calibration (Wildcards)	✅ Yes (`-ac`)	❌ No (Manual only)

SECTION 12 — CHEAT SHEETS & ONE-LINERS

FFUF Quick Reference

```
# Basic Directory Fuzzing
ffuf -w wordlist.txt -u http://target.com/FUZZ

# Auto-Calibrate (Automatically filter wildcard responses)
ffuf -w wordlist.txt -u http://target.com/FUZZ -ac

# Fuzzing File Extensions
ffuf -w wordlist.txt -u http://target.com/FUZZ -e .bak,.php,.txt

# Header Fuzzing (Auth Bypass)
ffuf -w wordlist.txt -u http://target.com/admin -H "X-Forwarded-For: FUZZ" -mc 200

# POST Data Fuzzing
ffuf -w wordlist.txt -X POST -d "user=admin&pass=FUZZ" -u http://target.com/login

# Filter out false positives by size (e.g. 42 bytes)
ffuf -w wordlist.txt -u http://target.com/FUZZ -fs 42
```

Gobuster Quick Reference

```
# Basic Directory Scan
gobuster dir -u http://target.com -w wordlist.txt

# Directory Scan with Extensions and ignore 404/403
gobuster dir -u http://target.com -w wordlist.txt -x php,txt,bak -b 404,403

# Subdomain DNS Brute Force
gobuster dns -d target.com -w subdomains.txt -t 50

# Virtual Host Discovery
gobuster vhost -u target.com -w subdomains.txt --append-domain

# S3 Bucket Hunting
gobuster s3 -w bucket_names.txt
```